

Dokumentacja projektu

Webowa platforma treningowa dla gitarzystów z lokalnym silnikiem audio

1. Executive Summary

1.1 Nazwa robocza projektu

Guitar Training Platform / Guitar Skill Engine

1.2 Streszczenie projektu

Projekt zakłada stworzenie profesjonalnej platformy treningowej dla gitarzystów, łączącej grywalizację, trening w czasie rzeczywistym oraz precyzyjną analizę gry użytkownika. Produkt ma działać w modelu hybrydowym:

- **warstwa webowa** odpowiada za interfejs, logikę produktową, progres użytkownika, content, ranking, płatności i doświadczenie użytkownika,
- **lokalny silnik audio** uruchamiany na komputerze użytkownika odpowiada za przechwytywanie sygnału z gitary, analizę dźwięku w czasie rzeczywistym, scoring i komunikację ze stroną.

Taki model pozwala połączyć zalety nowoczesnej aplikacji webowej z możliwościami natywnego przetwarzania audio o niskiej latencji.

1.3 Problem rynkowy

Obecne narzędzia dla gitarzystów najczęściej dzielą się na trzy grupy:

1. **aplikacje edukacyjne** – dobre pod kątem materiału, słabe pod kątem real-time feedback,
2. **oprogramowanie audio / DAW / pluginy** – mocne technicznie, ale mało angażujące i niedostosowane do regularnego treningu,
3. **gry rytmiczne** – angażujące, ale zwykle nieprzystosowane do prawdziwego instrumentu i profesjonalnego rozwoju umiejętności.

Projekt wypełnia lukę pomiędzy tymi segmentami: ma dać gitarzyście realny trening, mierzalny progres, wysoki poziom motywacji oraz nowoczesny i intuicyjny UX.

1.4 Główna wartość produktu

Produkt ma dostarczać użytkownikowi:

- natychmiastową informację zwrotną o poprawności gry,
- czytelny, mierzalny progres,
- angażującą formę codziennego treningu,
- system celów, wyzwań i ścieżek rozwoju,
- środowisko przypominające grę, a nie techniczne narzędzie audio.

2. Wizja produktu

2.1 Wizja długoterminowa

Stworzenie jednej z najbardziej angażujących i technicznie dopracowanych platform do treningu gitarowego, która stanie się codziennym narzędziem rozwoju dla gitarzystów elektrycznych i akustycznych.

2.2 Wizja doświadczenia użytkownika

Użytkownik ma odczuwać produkt jako połączenie:

- treningu umiejętności,
- gry muzycznej,
- systemu progresji,
- platformy rozwoju osobistego w grze na gitarze.

Docelowe doświadczenie:

1. użytkownik loguje się na stronę,
2. wybiera trening lub wyzwanie,
3. uruchamia sesję,
4. widzi czytelny interfejs przypominający grę,
5. gra na prawdziwej gitarze,
6. system ocenia jego wykonanie w czasie rzeczywistym,
7. po zakończeniu otrzymuje dokładny wynik i aktualizację progresu.

2.3 Pozycjonowanie produktu

Produkt powinien być pozycjonowany jako:

- premium,
- nowoczesny,
- technicznie zaawansowany,
- przystępny dla użytkownika,
- uzależniający w pozytywnym sensie dzięki dobrze zaprojektowanej pętli motywacyjnej.

3. Cele projektu

3.1 Cele biznesowe

Cele krótkoterminowe

- zbudowanie działającego MVP,
- walidacja zainteresowania użytkowników,
- uruchomienie pierwszej wersji subskrypcyjnej,
- pozyskanie pierwszych aktywnych testerów.

Cele średnioterminowe

- dopracowanie silnika audio i scoringu,
- zbudowanie silnej pętli retencyjnej,
- wdrożenie rankingu i challenge'y,
- rozbudowa biblioteki ćwiczeń i lekcji.

Cele długoterminowe

- zbudowanie rozpoznawalnej marki w segmencie guitar practice,
- rozwój społeczności użytkowników,
- skalowanie na kolejne rynki,
- rozwój wersji desktopowej i potencjalnie mobilnej,
- wprowadzenie bardziej zaawansowanej analizy wykonania.

3.2 Cele produktowe

- stworzenie systemu treningowego używanego regularnie, nie okazjonalnie,
- osiągnięcie wysokiego poziomu retencji poprzez grywalizację,
- zapewnienie wyraźnego poczucia progresu,
- zapewnienie wysokiej czytelności feedbacku,
- zminimalizowanie bariery wejścia dla użytkownika.

3.3 Cele techniczne

- niska latencja analizy audio,
- precyzyjne wykrywanie wysokości dźwięku i momentu ataku,
- stabilna synchronizacja pomiędzy stroną a silnikiem,
- modularna i skalowalna architektura,
- gotowość do dalszego rozwoju bez przepisywania całego systemu.

4. Zakres projektu

4.1 Zakres wersji MVP

MVP ma udowodnić, że model web + local engine działa i daje użytkownikowi wartość.

Zakres funkcjonalny MVP

- rejestracja i logowanie użytkownika,
- panel użytkownika,
- podstawowa ścieżka treningowa,
- ekran treningu z real-time feedback,
- integracja z lokalnym silnikiem audio,
- wykrywanie pojedynczych dźwięków,
- podstawowy scoring hit/miss + accuracy,
- zapisywanie wyników,
- proste statystyki sesji,
- podstawowy system progresu.

Zakres techniczny MVP

- frontend webowy,
- backend z bazą danych,
- lokalny companion app / engine,
- komunikacja localhost,
- obsługa minimum jednego trybu treningowego,
- podstawowy mechanizm aktualizacji i wersjonowania engine.

4.2 Funkcje poza MVP

- zaawansowana analiza artykulacji,
 - wykrywanie bendów, vibrato, slide'ów,
 - pełny multiplayer real-time,
 - mobilny companion,
 - rozbudowany marketplace contentu,
 - edytor własnych ćwiczeń dla użytkowników.
-

5. Grupa docelowa

5.1 Segmenty użytkowników

Segment A – początkujący i średniozaawansowani gitarzyści

Potrzebują: - motywacji, - struktury treningu, - prostego feedbacku, - poczucia progresu, - codziennej rutyny.

Segment B – ambitni hobbysci

Potrzebują: - mierzalności, - trudniejszych ćwiczeń, - porównywania wyników, - planowania rozwoju, - pracy nad precyzją i timingiem.

Segment C – zaawansowani użytkownicy

Potrzebują: - precyzyjnego scoringu, - rozbudowanych treningów, - czytelnych analiz błędów, - większej głębi systemu.

5.2 Kluczowe potrzeby użytkowników

- chcę ćwiczyć regularnie,
 - chcę widzieć postęp,
 - chcę wiedzieć, czy gram poprawnie,
 - chcę mieć cele,
 - chcę mieć trening, który nie nudzi,
 - chcę poczucia, że rozwijam realne umiejętności, a nie tylko klikam w aplikacji.
-

6. Założenia produktowe

6.1 Główne założenia

1. Produkt ma być odbierany jako **gra i platforma rozwoju**, nie jako narzędzie techniczne.
2. Użytkownik powinien spędzać większość czasu w aplikacji webowej.
3. Lokalny engine ma być prawie niewidoczny z perspektywy UX.
4. Silnik audio ma być źródłem prawdy dla detekcji i scoringu.
5. Interfejs webowy ma odpowiadać za prezentację, progres, flow i motywację.

6.2 Założenia UX

- ma być nowocześnie,
- ma być czytelnie,
- ma być szybko,
- ma być intuicyjnie,
- ma być możliwie mało „technicznie”.

6.3 Założenia techniczne

- przeglądarka nie powinna realizować precyzyjnej analizy audio,
 - komunikacja pomiędzy webem a engine musi być lokalna i szybka,
 - scoring nie powinien zależeć od zegara frontendu,
 - architektura musi umożliwiać późniejsze rozszerzenie engine bez przebudowy produktu.
-

7. Architektura rozwiązania

7.1 Uzasadnienie architektury hybrydowej

Architektura web + local engine wynika z realnych ograniczeń dwóch skrajnych podejść:

Dlaczego nie czysty web

- ograniczona kontrola nad audio,
- trudniejsza stabilizacja latencji,
- ograniczona przewidywalność między przeglądarkami i urządzeniami,
- większe ryzyko problemów z precyzją i synchronizacją.

Dlaczego nie pełny desktop app jako główny produkt

- wolniejszy development UI,
- trudniejszy onboarding,
- większa złożoność warstwy produktowej,
- gorszy marketing i dystrybucja,
- większy koszt iteracji UX.

Dlaczego model hybrydowy jest optymalny

- web daje szybkość iteracji i wysokiej jakości UX,

- lokalny engine daje kontrolę nad audio,
- warstwa produktowa i warstwa DSP są odseparowane,
- można rozwijać obie części niezależnie.

7.2 Warstwy systemu

Warstwa 1 – Frontend Web

Odpowiada za: - interfejs użytkownika, - ekran treningu, - dashboard, - nawigację, - komunikaty, - prezentację wyników, - część logiki produktowej.

Warstwa 2 – Backend / API / Baza danych

Odpowiada za: - użytkowników, - autoryzację, - dane sesji, - progres, - content, - rankingi, - analitykę.

Warstwa 3 – Local Audio Engine

Odpowiada za: - przechwytywanie wejścia audio, - przetwarzanie sygnału, - detekcję zdarzeń muzycznych, - scoring, - komunikację lokalną ze stroną.

8. Szczegółowa architektura modułów

8.1 Moduły aplikacji webowej

8.1.1 Auth Module

Zakres: - rejestracja, - logowanie, - reset hasła, - zarządzanie sesją, - status subskrypcji.

8.1.2 Profile Module

Zakres: - podstawowe dane użytkownika, - poziom, - historia aktywności, - ustawienia, - preferencje treningowe.

8.1.3 Dashboard Module

Zakres: - dzienne cele, - streak, - ostatnie treningi, - rekomendowane ćwiczenia, - podsumowanie progresu.

8.1.4 Progression Module

Zakres: - XP, - poziomy, - odblokowania, - ścieżka rozwoju, - milestone'y.

8.1.5 Content Module

Zakres: - ćwiczenia, - lekcje, - challenge'e, - kategorie umiejętności, - struktura materiału.

8.1.6 Training Session Module

Zakres: - uruchamianie sesji, - pobranie konfiguracji, - komunikacja z engine, - obsługa lifecycle sesji, - timeout, reconnect, error states.

8.1.7 Gameplay UI Module

Zakres: - timeline / note highway, - wskaźniki poprawności, - combo, - score, - countdown, - stan połączenia z engine.

8.1.8 Results Module

Zakres: - accuracy, - liczba trafień, - błędy, - wykres sesji, - aktualizacja progresu, - rekomendacje kolejnych kroków.

8.1.9 Leaderboard / Social Module

Zakres: - rankingi, - challenge'e społecznościowe, - porównania wyników, - sezonowe wydarzenia.

8.1.10 Billing Module

Zakres: - plany subskrypcji, - płatności, - status konta, - ograniczenia free/premium.

8.2 Moduły local engine

8.2.1 Device Management Module

Zakres: - wykrywanie urządzeń audio, - wybór wejścia, - poziom wejściowy, - sample rate, - buffer config, - test połączenia.

8.2.2 Audio Capture Module

Zakres: - przechwytywanie sygnału, - streaming próbek, - kontrola bufora, - ciągłość strumienia.

8.2.3 Preprocessing Module

Zakres: - noise gate, - filtering, - ewentualna normalizacja, - usuwanie oczywistych zakłóceń, - conditioning sygnału do dalszej analizy.

8.2.4 Onset Detection Module

Zakres: - wykrywanie początku dźwięku, - oznaczanie czasu ataku, - przekazywanie eventów do scoringu.

8.2.5 Pitch Detection Module

Zakres: - estymacja wysokości dźwięku, - stabilizacja wyniku, - confidence score, - mapowanie do nut muzycznych.

8.2.6 Timing & Evaluation Module

Zakres: - porównanie eventów użytkownika z oczekiwaną mapą nut, - obliczanie offsetu czasowego, - przypisanie klasy trafienia, - walidacja zgodności wykonania.

8.2.7 Scoring Module

Zakres: - perfect / good / miss, - combo, - accuracy, - final score, - ewentualny multiplier.

8.2.8 Session Runtime Module

Zakres: - start/stop sesji, - stan sesji, - wczytywanie konfiguracji, - obsługa clocka, - synchronizacja.

8.2.9 Communication Module

Zakres: - WebSocket / localhost API, - handshake, - streaming eventów, - heartbeat, - reconnect, - wersjonowanie protokołu.

8.2.10 Diagnostics Module

Zakres: - logowanie błędów, - latency test, - health status, - debug metrics.

9. Model komunikacji web ↔ local engine

9.1 Wymagania komunikacyjne

Komunikacja musi być: - lokalna, - szybka, - odporna na chwilowe zakłócenia, - przewidywalna, - łatwa do monitorowania, - bezpieczna.

9.2 Preferowany model

WebSocket na localhost.

Powody wyboru

- lepszy model do realtime niż ciągłe requesty HTTP,
- łatwe streamowanie zdarzeń,
- prosty mechanizm heartbeat,
- mniejsze opóźnienia logiczne,
- łatwiejsza obsługa sesji realtime.

9.3 Typy wiadomości

Wiadomości web → engine

- handshake/init,
- get_status,
- get_devices,

- select_device,
- calibrate,
- start_session,
- pause_session,
- resume_session,
- stop_session,
- ping.

Wiadomości engine → web

- ready,
- status,
- devices,
- calibration_result,
- session_started,
- note_event,
- timing_result,
- score_update,
- error,
- warning,
- heartbeat.

9.4 Propozycja struktury payloadów

Wszystkie wiadomości powinny posiadać: - `type`, - `sessionId`, - `timestamp`, - `version`, - `payload`.

Przykład:

```
{
  "type": "note_event",
  "sessionId": "abc123",
  "timestamp": 1712345678901,
  "version": 1,
  "payload": {
    "detectedPitchHz": 329.63,
    "noteName": "E4",
    "confidence": 0.92,
    "onsetMs": 3245,
    "timingOffsetMs": -18,
    "judgement": "good"
  }
}
```

9.5 Zasady komunikacji

- frontend nie interpretuje surowych danych bez potrzeby,
- scoring jest liczony przez engine,
- web renderuje wynik,

- krytyczne eventy mają potwierdzenie odbioru, jeśli to potrzebne,
 - błędy komunikacji muszą być jawnie obsługiwane w UI.
-

10. Synchronizacja czasu

10.1 Główny problem

Największym ryzykiem w tego typu systemie nie jest samo wykrycie dźwięku, lecz poprawne przypisanie go do momentu na osi czasu sesji.

10.2 Zasada architektoniczna

Engine jest źródłem prawdy czasowej dla oceny muzycznej.

Frontend nie może być ostatecznym źródłem prawdy dla scoringu, ponieważ: - render UI nie jest deterministyczny, - browser loop nie gwarantuje stabilnej precyzji, - frame drops nie mogą wpływać na wynik użytkownika.

10.3 Model synchronizacji

1. sesja startuje z jasno zdefiniowanym czasem startowym,
2. note map zawiera oczekiwane timestamps,
3. engine używa własnego clocka do oceny,
4. frontend renderuje stan na podstawie eventów i synchronizacji z sesją,
5. wynik końcowy jest autorytatywnie liczony przez engine.

10.4 Okna trafień

Należy wprowadzić predefiniowane okna trafień, np.: - Perfect, - Good, - Miss.

Wartości należy trzymać konfigurowalnie, a nie zaszyte na sztywno w UI.

11. Silnik audio – podejście techniczne

11.1 Wymagania podstawowe

Silnik ma: - działać stabilnie na komputerze użytkownika, - obsługiwać wejście z interfejsu audio, - mieć możliwie małą latencję, - wykrywać pojedyncze dźwięki i momenty ich zagrania, - odsyłać gotowe eventy do frontendu.

11.2 Wymagania jakościowe

- brak zawiesznień przy dłuższej pracy,
- odporność na chwilowe skoki sygnału,
- czytelna diagnostyka,
- sensowna jakość wyników już w MVP,

- możliwość iteracyjnej poprawy algorytmów bez zmiany całej architektury.

11.3 Zakres DSP w MVP

- podstawowy preprocessing,
- onset detection,
- pitch detection dla nut pojedynczych,
- confidence filtering,
- scoring w oparciu o note map.

11.4 Zakres DSP po MVP

- lepsza odporność na szum,
 - analiza sustainu,
 - stabilizacja pitchu w trudniejszych przypadkach,
 - bardziej zaawansowana analiza technik gry.
-

12. Technologie i uzasadnienie wyboru

12.1 Frontend

Rekomendacja

- Next.js
- TypeScript
- Tailwind CSS
- component-based architecture
- canvas/SVG, opcjonalnie PixiJS dla gameplay renderingu

Uzasadnienie

- szybki development,
- dobra organizacja aplikacji produktowej,
- łatwa rozbudowa paneli i dashboardu,
- wysoka kontrola nad UI,
- dobra zgodność z nowoczesnym stackiem webowym.

12.2 Backend

Rekomendacja

Jedna z dwóch dróg:

Opcja A – Supabase

- szybki start,
- auth,
- baza,
- storage,

- realtime tam gdzie potrzebny,
- dobry wybór dla MVP.

Opcja B – Własny backend

- Node.js / NestJS / Fastify,
- PostgreSQL,
- własna warstwa auth i API.

Uzasadnienie

- Supabase przyspiesza MVP,
- custom backend daje większą kontrolę przy wzroście skali.

12.3 Local Engine

Rekomendacja główna

- Rust lub C++ dla warstwy audio i DSP.

Rekomendacja praktyczna

- Tauri jako warstwa desktopowa / wrapper + Rust w rdzeniu.

Uzasadnienie

- możliwość zbudowania lekkiej aplikacji lokalnej,
- dobra wydajność,
- czytelna struktura,
- łatwiejsza kontrola nad lokalnym runtime niż w przeglądarce.

12.4 Komunikacja

- WebSocket na localhost,
- opcjonalne HTTP endpoints do prostych operacji diagnostycznych.

12.5 Monitoring i logging

- logi lokalne po stronie engine,
- logi aplikacyjne po stronie backendu,
- centralne error tracking dla web app.

13. Proponowana struktura repozytorium

13.1 Monorepo

Rekomendowane jest monorepo.

Przykładowa struktura

```
/apps
  /web
  /backend
  /engine-shell
/packages
  /ui
  /shared-types
  /protocol
  /core-domain
  /content-schema
/engine
  /audio-core
  /dsp
  /session-runtime
/docs
  /product
  /architecture
  /api
  /protocol
  /qa
```

13.2 Uzasadnienie monorepo

- wspólne typy,
 - wspólny protokół,
 - spójność wersjonowania,
 - łatwiejsza kontrola nad integracją między webem i engine.
-

14. Model domenowy produktu

14.1 Główne encje

- User
- UserProfile
- Subscription
- SkillTrack
- Lesson
- Exercise
- SessionTemplate
- TrainingSession
- SessionResult
- ProgressState
- LeaderboardEntry
- EngineStatus
- AudioDevice

14.2 Kluczowe relacje

- User posiada UserProfile,
 - User posiada wiele SessionResult,
 - Lesson zawiera Exercises,
 - Exercise może należeć do SkillTrack,
 - TrainingSession uruchamiana jest na podstawie SessionTemplate,
 - SessionResult aktualizuje ProgressState.
-

15. Model danych – zakres minimalny

15.1 Użytkownik

- id,
- email,
- auth provider,
- createdAt,
- subscriptionTier,
- accountStatus.

15.2 Profil

- displayName,
- avatar,
- skillLevel,
- preferredGenres,
- preferredPracticeDuration,
- onboardingCompleted.

15.3 Ćwiczenie

- id,
- title,
- description,
- difficulty,
- category,
- bpm,
- noteMap,
- tags,
- estimatedDuration.

15.4 Sesja treningowa

- id,
- userId,
- templateId,
- startedAt,
- endedAt,

- result,
 - accuracy,
 - streak,
 - score,
 - notesHit,
 - notesMissed,
 - engineVersion.
-

16. Content system

16.1 Dlaczego content system jest krytyczny

Silnik i UI nie wystarczą. Produkt musi dostarczać treści treningowe w sposób spójny, skalowalny i łatwy do rozbudowy.

16.2 Wymagania wobec contentu

- modularność,
- tagowanie,
- kategoryzacja według umiejętności,
- poziomy trudności,
- możliwość personalizacji,
- możliwość rekomendacji kolejnych ćwiczeń.

16.3 Struktura contentu

Rekomendowane poziomy: - Path / Track, - Lesson, - Exercise, - Session Template, - Challenge.

16.4 Note map

Każde ćwiczenie powinno posiadać formalny opis tego, co ma zostać zagrane.

Minimalne pola: - timestamp, - expectedNote, - lane/string, jeśli dotyczy, - allowedWindow, - optional metadata.

17. Gameplay design

17.1 Cel gameplayu

Gameplay nie ma imitować klasycznej gry zręcznościowej 1:1, ale ma dawać użytkownikowi: - jasny cel, - czytelne zadania, - napięcie, - feedback, - satysfakcję z poprawnego wykonania.

17.2 Komponenty gameplayu

- countdown,
- lane/timeline,

- target events,
- hit indicators,
- combo,
- score,
- pasek progresu,
- końcowe podsumowanie.

17.3 Zasady projektowe

- maksymalna czytelność,
 - brak przeładowania ekranów,
 - feedback ma być natychmiastowy,
 - użytkownik nie może zastanawiać się, co ma zrobić dalej,
 - błędy powinny być czytelne, ale nie frustrujące.
-

18. Progression i retencja

18.1 Cel systemu progresji

Doprowadzić do tego, by użytkownik wracał do produktu regularnie, bo: - widzi wzrost, - ma cele, - ma streak, - odblokowuje nowe rzeczy, - porównuje się z innymi lub z samym sobą.

18.2 Podstawowe mechanizmy

- XP,
- poziomy,
- daily goals,
- streak,
- weekly challenges,
- lesson path,
- milestone rewards.

18.3 Zasada projektowa

Progres ma wynikać z realnego treningu, a nie z pustych animacji i sztucznego nabijania punktów.

19. Bezpieczeństwo

19.1 Zakres bezpieczeństwa

- auth i sesje użytkownika,
- bezpieczna komunikacja backendowa,
- ograniczenie ryzyka nadużycia localhost engine,
- walidacja danych wejściowych,
- ochrona wyników i integralności progresu.

19.2 Ryzyka

- podszywanie się pod engine,
- manipulowanie wynikami,
- nieuprawnione wywołania lokalnego API,
- niespójność wersji klienta i engine.

19.3 Minimalne zabezpieczenia

- handshake z tokenem sesyjnym,
 - whitelistowane.originy komunikacji lokalnej,
 - wersjonowanie protokołu,
 - walidacja payloadów,
 - sanity checks na wynikach sesji.
-

20. Wersjonowanie i kompatybilność

20.1 Problem

Web i engine będą rozwijane niezależnie, więc wersjonowanie musi być zaprojektowane od początku.

20.2 Zasady

- protokół komunikacyjny ma własny numer wersji,
- engine zgłasza swoją wersję do web app,
- frontend może wymagać minimalnej wersji engine,
- krytyczne zmiany muszą być obsługiwane jawnie.

20.3 Strategia

- semver dla komponentów,
 - compatibility matrix w dokumentacji,
 - fallback UI w przypadku niezgodności wersji.
-

21. Instalacja i onboarding użytkownika

21.1 Cel

Mimo architektury hybrydowej proces wejścia użytkownika musi być prosty i klarowny.

21.2 Pożądany flow

1. użytkownik tworzy konto,
2. pobiera local engine,
3. uruchamia aplikację w tle,
4. wybiera urządzenie wejściowe,

5. przechodzi prostą kalibrację,
6. wraca na stronę i rozpoczyna trening.

21.3 Krytyczne elementy onboardingu

- jasna instrukcja instalacji,
 - wykrywanie statusu engine,
 - prosty test wejścia audio,
 - krótka kalibracja,
 - brak przeciążenia informacyjnego.
-

22. QA i testowanie

22.1 Zakres testów

Web

- testy komponentów,
- testy integracyjne,
- testy przepływów użytkownika,
- testy stanów błędów,
- testy kompatybilności przeglądarek.

Backend

- testy API,
- testy autoryzacji,
- testy modelu danych,
- testy integralności wyników.

Engine

- testy urządzeń audio,
- testy stabilności runtime,
- testy algorytmów detekcji,
- testy latencji,
- testy reconnect i recovery.

End-to-end

- start sesji,
- komunikacja web ↔ engine,
- zapisywanie wyników,
- obsługa błędów.

22.2 Priorytety QA

Najwyższy priorytet: - niezawodność sesji, - poprawność scoringu, - brak fałszywie pozytywnych wyników, - jasna obsługa awarii.

23. Monitoring i obserwowalność

23.1 Co monitorować

- błędy frontendu,
- błędy backendu,
- błędy połączenia z engine,
- czas startu sesji,
- utratę połączenia,
- skuteczność ukończenia sesji,
- aktywność użytkownika,
- skuteczność onboarding flow.

23.2 Kluczowe wskaźniki produktowe

- activation rate,
- onboarding completion,
- daily active users,
- weekly retention,
- session completion rate,
- average session length,
- conversion to paid.

23.3 Kluczowe wskaźniki techniczne

- average session start time,
 - error rate per session,
 - engine connection success rate,
 - reconnect frequency,
 - crash rate,
 - audio device initialization success.
-

24. Roadmapa realizacji

24.1 Faza 0 – Discovery i architektura

Cele: - doprecyzowanie wizji, - wybór stacku, - zdefiniowanie MVP, - przygotowanie dokumentacji domenowej i technicznej, - proof of concept komunikacji.

Deliverables: - dokument architektury, - model domeny, - protokół komunikacyjny v1, - backlog MVP.

24.2 Faza 1 – Proof of Concept

Cele: - uruchomienie prostego engine, - lokalna komunikacja ze stroną, - wykrycie pojedynczych dźwięków, - prosty ekran treningu.

Deliverables: - demo web ↔ engine, - start session, - live note events, - podstawowy hit/miss.

24.3 Faza 2 – MVP Product

Cele: - auth, - dashboard, - ćwiczenia, - wyniki, - progres, - podstawowa subskrypcja, - podstawowy onboarding.

Deliverables: - publiczna wersja testowa, - kompletna podstawowa ścieżka użytkownika, - telemetria i logi.

24.4 Faza 3 – Stabilizacja

Cele: - poprawa jakości scoringu, - poprawa UX, - poprawa onboarding flow, - optymalizacja komunikacji, - redukcja błędów.

24.5 Faza 4 – Rozszerzenia

Cele: - więcej trybów treningu, - lepsza analityka, - challenge'e społeczne, - bardziej zaawansowany content, - dodatkowe funkcje premium.

25. Backlog funkcjonalny – priorytety

25.1 P0 – absolutnie krytyczne

- lokalny engine działa,
- web wykrywa engine,
- można uruchomić sesję,
- wykrywana jest nuta,
- wynik jest wyświetlany,
- wynik zapisuje się w backendzie.

25.2 P1 – bardzo ważne

- dashboard,
- ścieżka progresu,
- lepszy wynik końcowy,
- konfiguracja wejścia audio,
- podstawowa kalibracja,
- błędy i reconnect.

25.3 P2 – ważne po MVP

- rozbudowane statystyki,
 - challenge'e tygodniowe,
 - rankingi,
 - większa liczba trybów treningu.
-

26. Ryzyka projektowe

26.1 Ryzyka techniczne

- niestabilność detekcji audio,
- problemy z różnymi urządzeniami,
- trudności w synchronizacji,
- zbyt wysoka latencja w niektórych konfiguracjach,
- złożoność aktualizacji engine.

26.2 Ryzyka produktowe

- zbyt skomplikowany onboarding,
- zbyt mała motywacja do powrotów,
- zbyt techniczny UX,
- rozjazd między obietnicą marketingową a realnym doświadczeniem.

26.3 Ryzyka organizacyjne

- zbyt szeroki scope na start,
- brak priorytetyzacji,
- zbyt wczesna rozbudowa funkcji pobocznych,
- skupienie na detalach zamiast na działającym rdzeniu produktu.

26.4 Strategie ograniczania ryzyk

- silne MVP discipline,
 - jasny podział priorytetów,
 - testy na realnym sprzęcie od początku,
 - telemetry-first approach,
 - szybka iteracja po feedbacku użytkowników.
-

27. Zasady realizacji projektu

27.1 Główne zasady

1. Najpierw działa rdzeń, potem rozbudowa.
2. UI nie może wymuszać złych decyzji architektonicznych.
3. Engine musi pozostać niezależny od warstwy wizualnej.
4. Protokół komunikacji ma być jawny i wersjonowany.
5. Wszystkie istotne decyzje muszą mieć uzasadnienie.
6. Funkcje niekrytyczne nie mogą blokować MVP.

27.2 Zasady dla developmentu

- małe iteracje,
- jasne definition of done,
- osobne milestone'y dla webu i engine,

- każda funkcja musi mieć właściciela domenowego,
 - dokumentacja aktualizowana wraz z rozwojem systemu.
-

28. Definition of Done dla MVP

MVP można uznać za gotowe, gdy: - użytkownik tworzy konto, - instaluje engine, - przechodzi onboarding, - wybiera trening, - sesja uruchamia się poprawnie, - system wykrywa zagrane dźwięki, - użytkownik otrzymuje real-time feedback, - wynik zapisuje się i aktualizuje progres, - doświadczenie jest wystarczająco stabilne, by oddać je pierwszym testerom.

29. Dokumenty uzupełniające do stworzenia

Na bazie tej dokumentacji należy przygotować osobne dokumenty robocze:

1. **Product Requirements Document (PRD)**
 2. **Architecture Decision Records (ADR)**
 3. **Protocol Specification**
 4. **Engine Technical Specification**
 5. **Frontend Technical Specification**
 6. **Backend/API Specification**
 7. **Content Model Specification**
 8. **Onboarding Flow Specification**
 9. **QA Test Plan**
 10. **Release Plan MVP**
-

30. Podsumowanie strategiczne

To nie jest zwykła strona webowa i nie jest to też klasyczna aplikacja desktopowa. Jest to produkt hybrydowy, którego sukces zależy od poprawnego rozdzielenia odpowiedzialności:

- web ma sprzedawać doświadczenie, progres i codzienny flow,
- engine ma dostarczać techniczną prawdę o grze użytkownika.

Jeżeli ten podział zostanie utrzymany od początku, projekt ma solidne fundamenty do zbudowania profesjonalnego produktu.

Najważniejszy priorytet na starcie: **zbudować działający, stabilny i wiarygodny rdzeń treningowej sesji realtime.**

Reszta produktu ma ten rdzeń wzmacniać, a nie go zastępować.